

React Interview Prep

Topic 13 — React Router: Complete Guide

Setup · Link · useNavigate · Dynamic Routes · Query Params · Nested Routes · Protected Routes
· Lazy Loading · All Hooks

- Covers All examples from conversation — nothing skipped
- Series React Interview Prep — Session 13

by Akshay Chavhan • React Interview Series

1. Like You're 5 — TV Channels

Imagine a TV with multiple channels. Channel 1 = Home, Channel 2 = About, Channel 3 = Profile. When you press a channel button — the TV doesn't restart. It just swaps what's showing while everything else stays the same. React Router does exactly this — swaps components based on the URL without reloading the page.

■ **React Router = client-side routing. URL changes → component swaps → NO page reload.**
Users can bookmark links, press back/forward, and share URLs.

2. Installation & Basic Setup

```
npm install react-router-dom

import { BrowserRouter, Routes, Route } from 'react-router-dom';
function App() {
  return (
    <BrowserRouter> { /* wraps entire app – provides routing context */}
    <Routes> { /* looks at URL, renders matching Route */}
    <Route path="/" element={<HomePage />} />
    <Route path="/about" element={<AboutPage />} />
    <Route path="/profile" element={<ProfilePage />} />
    <Route path="*" element={<NotFoundPage />} />
    { /* ↑ wildcard – catches ALL unmatched routes */}
    </Routes>
  </BrowserRouter>
);
}
```

3. Link vs NavLink — Navigation in JSX

```
import { Link, NavLink } from 'react-router-dom';
function Navbar() {
  return (
    <nav>
      { /* Link – basic navigation, no page reload */}
      <Link to="/">Home</Link>
      <Link to="/about">About</Link>
      { /* NavLink – adds 'active' class when route matches */}
      <NavLink
        to="/profile"
        style={({ isActive }) => ({
          color: isActive ? 'blue' : 'black',
          fontWeight: isActive ? 'bold' : 'normal',
        })}
      >
        Profile
      </NavLink>
    </nav>
  );
}
```

	Link	NavLink
--	------	---------

Purpose	Basic navigation	Navigation + active state styling
Active class	■ No	■ Yes — isActive prop available
Use for	Simple nav links	Navbars where active item needs styling

4. useNavigate — Programmatic Navigation

When you need to navigate after an action (login, form submit, delete) — not from a click on a Link.

```
import { useNavigate } from 'react-router-dom';

function LoginPage() {
  const navigate = useNavigate();
  async function handleLogin() {
    await loginUser();
    navigate('/dashboard'); // go forward ■
    navigate(-1); // back button ■
    navigate('/login', { replace: true }); // no back history ■
  }
  return <button onClick={handleLogin}>Login</button>;
}
```

■ Wrong

```
// ■ navigate() – history stack grows
// User CAN press back
navigate('/dashboard');

// Problem after logout:
// User presses back → goes to
// protected dashboard ■
```

■ Correct

```
// ■ replace – replaces current entry
// User CANNOT press back
navigate('/login', { replace: true });

// After logout → login page replaces
// no way back to protected route ■
```

5. Dynamic Routes — URL Parameters

Use `:` prefix to define a dynamic segment in the URL. Read it with `useParams()`.

```
// Define dynamic route
<Route path='/users/:userId' element={<UserProfile />} />
<Route path='/products/:category/:id' element={<ProductDetail />} />

// Read params inside the component
import { useParams } from 'react-router-dom';

function UserProfile() {
  const { userId } = useParams();
  // ↑ matches :userId from the route path
  const { data } = useFetch(`/api/users/${userId}`);
  return <h1>{data?.name}</h1>;
}

// Navigate to dynamic route
<Link to='/users/123'>View User</Link>
<Link to={`/${users}/${user.id}`}>View {user.name}</Link>
```

6. Query Parameters — Search & Filter State in URL

Query params like `?category=phones&sort=price` — readable and writable with `useSearchParams()`.

```
import { useSearchParams } from 'react-router-dom';

function ProductsPage() {
  const [searchParams, setSearchParams] = useSearchParams();
  // Read: /products?category=phones&sort=price
  const category = searchParams.get('category'); // 'phones'
  const sort = searchParams.get('sort'); // 'price'
  function handleFilter(newCategory) {
    setSearchParams({ category: newCategory, sort });
    // URL updates → component re-renders with new params ■
  }
  return (
    <div>
      <button onClick={() => handleFilter('phones')}>Phones</button>
      <button onClick={() => handleFilter('laptops')}>Laptops</button>
      <p>Showing: {category}</p>
    </div>
  );
}
```

■ **Query params keep filter/sort state IN the URL — users can share/bookmark the exact filtered view.**

7. Nested Routes — Shared Layouts

Parent route provides a layout (sidebar, navbar). Child routes render inside it via `<Outlet />`.

```
// Layout component – shared UI across child routes
function DashboardLayout() {
  return (
    <div>
      <Sidebar /> { /* always visible */}
      <Topbar /> { /* always visible */}
      <Outlet /> { /* ← child route renders HERE */}
    </div>
  );
}

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path='/dashboard' element={<DashboardLayout />} />
        <Route index element={<DashboardHome />} /> { /* /dashboard */}
        <Route path='stats' element={<Stats />} /> { /* /dashboard/stats */}
        <Route path='users' element={<Users />} /> { /* /dashboard/users */}
      </Routes>
    </BrowserRouter>
  );
}
```

■ *Outlet is the placeholder where child routes render. Without Outlet in the parent, child routes have nowhere to appear.*

8. Protected Routes — Auth Guard

Restrict access to authenticated users only. Redirect to login if not logged in.

```
function ProtectedRoute({ children }) {
  const { user } = useAuth();
  if (!user) {
    return <Navigate to='/login' replace />; // not logged in → redirect
  }
  return children; // logged in → render the page ■
}

// Usage in App – wrap any sensitive route
<Route path='/dashboard' element={
  <ProtectedRoute>
    <Dashboard />
  </ProtectedRoute>
} />

<Route path='/settings' element={
  <ProtectedRoute>
    <Settings />
  </ProtectedRoute>
} />
```

9. Lazy Loading Routes — Code Splitting

Each page loads only when visited. Reduces initial bundle size — faster first load.

■ Wrong

```
// ■ Without lazy
// ALL pages download on first load
import HomePage from
  './pages/HomePage';

import AboutPage from
  './pages/AboutPage';

import ProfilePage from
  './pages/ProfilePage';

// User visits only homepage
// but downloaded all 3 pages
```

■ Correct

```
// ■ With lazy – on-demand loading
const HomePage = lazy(()=>import(
  './pages/HomePage'));

const AboutPage = lazy(()=>import(
  './pages/AboutPage'));

const ProfilePage = lazy(()=>import(
  './pages/ProfilePage'));

// User visits homepage
// only HomePage chunk downloads ■
```

```
function App() {
  return (
    <BrowserRouter>
    <Suspense fallback={<div>Loading...</div>}>
    <Routes>
    <Route path="/" element={<HomePage />} />
    <Route path="/about" element={<AboutPage />} />
    <Route path="/profile" element={<ProfilePage />} />
    </Routes>
    </Suspense>
    </BrowserRouter>
  );
}
```

10. useLocation — Read Current URL Object

```
import { useLocation } from 'react-router-dom';

function App() {
  const location = useLocation();
  // location = {
  //   pathname: '/products',
  //   search: '?category=phones',
  //   hash: '#reviews',
  //   state: { from: '/home' }
  // }

  // Track page views on route change
  useEffect(() => {
    analytics.pageView(location.pathname);
  }, [location]);

  // Pass state between routes
  navigate('/dashboard', { state: { from: '/checkout' } });

  // Read state in destination
  const location = useLocation();
  const cameFrom = location.state?.from; // '/checkout'
}
```

11. All React Router Hooks — Quick Reference

Hook	Returns	Common Use Case
useNavigate()	navigate function	Go to route after login/submit/delete
useParams()	{ param: value }	Read :userId from /users/:userId
useSearchParams()	[params, setParams]	Read/write ?category=phones in URL
useLocation()	location object	Track page views, pass state between routes
useMatch(pattern)	match object or null	Check if current URL matches a pattern

12. BrowserRouter vs HashRouter

■ Wrong

```
// BrowserRouter – clean URLs
// needs server to redirect all
// routes
// to index.html
//
// http://myapp.com/dashboard ■
// http://myapp.com/users/123 ■
//
// Works on: Vercel, Netlify,
// any server with redirect config
```

■ Correct

```
// HashRouter – uses # in URL
// works WITHOUT server config
//
// http://myapp.com/#/dashboard
// http://myapp.com/#/users/123
//
// Use for: GitHub Pages,
// static hosts that cannot
// redirect
// all routes to index.html
```

■ **Use BrowserRouter always. Only use HashRouter for GitHub Pages or static hosts that cannot configure server-side redirects.**

13. Complete Flow Diagram



14. Interview Q&A;

Q

Beginner

Q1. What is React Router and why do we need it?

→ React Router is a library for client-side routing in React apps. Without it, navigating between views requires a full page reload. React Router swaps components based on the URL without reloading — giving a fast app-like experience. It also syncs the URL so users can bookmark and share links.

Q

Beginner

Q2. What is the difference between Link and useNavigate?

→ Link renders an anchor tag for declarative navigation in JSX — used in navbars and menus. useNavigate returns a function for programmatic navigation in event handlers or after async operations like login. Both prevent full page reloads — they serve different use cases.

Q

Beginner

Q3. What are dynamic routes and how do you read params?

→ Dynamic routes use a colon prefix like `/users/:userId`. The `:userId` part matches any value in that URL segment. You read it in the component using `useParams` which returns an object with all matched params — `const { userId } = useParams()`.

Q

Intermediate

Q4. What is an Outlet in React Router?

→ Outlet is a placeholder component used in layout routes. When a parent route has child routes, Outlet marks where the matched child route's component will render. Without Outlet, child routes have no place to appear in the parent layout.

Q

Intermediate

Q5. What are protected routes and how do you implement them?

→ Protected routes restrict access to authenticated users. You create a wrapper component that checks auth state — if not logged in it redirects using the `Navigate` component, if logged in it renders the children. You then wrap any sensitive route with this `ProtectedRoute` component.

Q

Intermediate

Q6. What is the difference between navigate and navigate with replace?

→ `navigate('/path')` pushes a new entry onto the browser history stack — users can press back to return. `navigate('/path', { replace: true })` replaces the current entry — users cannot press back. Use `replace` after login/logout so users cannot navigate back into protected pages.

Q**Expert****Q7. What is lazy loading routes and why use it?**

→ Lazy loading uses `React.lazy` and `dynamic import()` to split each page into a separate chunk. The chunk only downloads when the user visits that route — reducing initial bundle size and making the app load faster. Always wrap lazy routes in `Suspense` with a fallback UI.

Q**Expert****Q8. What is the difference between `useParams` and `useSearchParams`?**

→ `useParams` reads dynamic path segments like `:userId` from the route definition — `/users/123` gives `{ userId: '123' }`. `useSearchParams` reads and writes query string parameters like `?category=phones` — it returns a state-like pair `[params, setParams]` so updating params triggers a re-render.